

Please create the Arduino code for a class that contains an enum and a get-function that returns the enum names. The enum names are the uppercase letters "A" to "H" and they are stored in an x-macro. The x-macro is used to populate the enum and the get-function.



Hier ist ein kompakter Entwurf für eine Klasse, die ein X-Macro nutzt. Das Makro definiert sowohl die Enum-Werte als auch die zugehörigen Strings, was den Code wartungsfreundlich macht.

cpp

```
// 1. Das X-Macro: Liste der Werte
#define LETTER_LIST(X) \
    X(A) \
    X(B) \
    X(C) \
    X(D) \
    X(E) \
    X(F) \
    X(G) \
    X(H)

class LetterManager {
public:
    // 2. Enum über das Makro generieren
    enum Letter {
        #define AS_ENUM(name) name,
        LETTER_LIST(AS_ENUM)
        #undef AS_ENUM
    };

    // 3. Funktion zur Rückgabe des Namens als String
    static const char* getName(Letter l) {
        switch (l) {
            #define AS_CASE(name) case name: return #name;
            LETTER_LIST(AS_CASE)
            #undef AS_CASE
            default: return "Unknown";
        }
    }
};

void setup() {
    Serial.begin(9600);

    // Beispiel-Aufruf
    LetterManager::Letter myLetter = LetterManager::C;

    Serial.print("Enum Wert: ");
    Serial.println(myLetter); // Gibt den Index (2) aus

    Serial.print("Enum Name: ");
    Serial.println(LetterManager::getName(myLetter)); // Gibt "C" aus
}

void loop() {}
```

```
void loop() {
```

Verwende Code mit Vorsicht.

Wie es funktioniert:

- **Wartbarkeit:** Wenn du ein "I" hinzufügen möchtest, musst du es nur einmal oben im `LETTER_LIST` Makro eintragen.
- **#name (Stringification):** Im `AS_CASE` Teil sorgt das `#` dafür, dass der Enum-Name direkt in einen String umgewandelt wird.
- **Switch-Case:** Das Makro entfaltet sich innerhalb der `getName`-Funktion zu einer vollständigen Liste von `case A: return "A";` usw.

Möchtest du die Namen lieber in einem **Array im Flash-Speicher (PROGMEM)** ablegen, um RAM zu sparen?

KI-Antworten können Fehler enthalten. [Weitere Informationen](#)

```

    }
    }
    }
    }

```

The result looks good. Please modify it so that the letter strings are stored in PROGMEM.

Um die Strings im **PROGMEM** (Flash-Speicher) zu speichern, müssen wir die String-Literale explizit als `prog_char` Arrays anlegen. Da ein einfaches Array von Strings im Flash etwas tricky ist (man benötigt ein Array von Zeigern, die selbst auf Flash-Adressen zeigen), ist hier der effizienteste Weg über das X-Macro:

cpp

```
// 1. Das X-Macro
#define LETTER_LIST(X) \
    X(A) X(B) X(C) X(D) X(E) X(F) X(G) X(H)

// 2. Die einzelnen Strings im Flash (PROGMEM) definieren
#define AS_PROGMEM_STR(name) const char str_##name[] PROGMEM = #name;
LETTER_LIST(AS_PROGMEM_STR)
#undef AS_PROGMEM_STR

class LetterManager {
public:
    enum Letter {
        #define AS_ENUM(name) name,
        LETTER_LIST(AS_ENUM)
        #undef AS_ENUM
    };

    // 3. Ein Array von Zeigern im Flash, die auf die Flash-Strings zeigen
    static const char* const nameTable[] PROGMEM;

    static const __FlashStringHelper* getName(Letter l) {
        // Zugriff auf die Zeigertabelle im Flash
    }
};
```

```

        return (const __FlashStringHelper*)pgm_read_ptr(&nameTable[1]);
    }
};

// Definition der Zeigertabelle im Flash
const char* const LetterManager::nameTable[] PROGMEM = {
    #define AS_TABLE(name) str_##name,
    LETTER_LIST(AS_TABLE)
    #undef AS_TABLE
};

void setup() {
    Serial.begin(9600);

    LetterManager::Letter myLetter = LetterManager::E;

    Serial.print(F("Enum Name aus PROGMEM: "));
    // getName gibt einen __FlashStringHelper* zurück, den Serial.print versteht
    Serial.println(LetterManager::getName(myLetter));
}

void loop() {}

```

Verwende Code mit Vorsicht.

Was hat sich geändert?

- **Indirektion:** Wir speichern zuerst jeden Namen als einzelnes `char[]` im Flash. Danach erstellen wir ein Array (`nameTable`), das die Adressen dieser Namen enthält – ebenfalls im Flash.
- `__FlashStringHelper*`: Dies ist ein spezieller Arduino-Typ. Er sagt `Serial.print`, dass der String nicht im RAM, sondern im Flash liegt. So sparst du wertvollen Arbeitsspeicher.
- `pgm_read_ptr`: Da die Tabelle selbst im Flash liegt, müssen wir die Adresse des Strings mit dieser Funktion "herausfischen", bevor wir sie verwenden können.

Soll ich die Klasse so erweitern, dass du auch **per String nach dem Enum suchen** kannst?

KI-Antworten können Fehler enthalten. [Weitere Informationen](#)

yes

Um eine Suchfunktion hinzuzufügen, die einen String (`const char*`) entgegennimmt und den passenden Enum-Wert zurückgibt, erweitern wir die Klasse um eine `getEnum`-Methode. Da die Namen im Flash (PROGMEM) liegen, nutzen wir `strcmp_P`, um den Eingabestring mit den Flash-Strings zu vergleichen.

Hier ist der vollständige Code:

cpp

```

#include <Arduino.h>

// 1. Das X-Macro
#define LETTER_LIST(X) \
    X(A) X(B) X(C) X(D) X(E) X(F) X(G) X(H)

// 2. Strings im Flash (PROGMEM)
#define AS_PROGMEM_STR(name) const char str_##name[] PROGMEM = #name;
LETTER_LIST(AS_PROGMEM_STR)
#undef AS_PROGMEM_STR

class LetterManager {
public:
    enum Letter {
        #define AS_ENUM(name) name,
        LETTER_LIST(AS_ENUM)
        #undef AS_ENUM
        UNKNOWN = -1
    };

    static const char* const nameTable[] PROGMEM;
    static const uint8_t count = 8; // Anzahl der Elemente

    // Rückgabe des Namens (Flash -> Serial)
    static const __FlashStringHelper* getName(Letter l) {
        if (l < 0 || l >= count) return F("Unknown");
        return (const __FlashStringHelper*)pgm_read_ptr(&nameTable[l]);
    }

    // NEU: Suche nach dem Enum-Wert anhand eines Strings
    static Letter getEnum(const char* searchName) {
        for (uint8_t i = 0; i < count; i++) {
            // Hole die Adresse des Namens aus der Tabelle
            const char* namePtr = (const char*)pgm_read_ptr(&nameTable[i]);
            // Vergleiche RAM-String mit PROGMEM-String
            if (strcmp_P(searchName, namePtr) == 0) {
                return (Letter)i;
            }
        }
        return UNKNOWN;
    }
};

// Definition der Zeigertabelle
const char* const LetterManager::nameTable[] PROGMEM = {
    #define AS_TABLE(name) str_##name,
    LETTER_LIST(AS_TABLE)
    #undef AS_TABLE
};

void setup() {
    Serial.begin(9600);
    while(!Serial);

    // Test 1: Von Enum zu Name
    LetterManager::Letter myLetter = LetterManager::D;
    Serial.print(F("Name von Index 3: "));

```

```
Serial.println(LetterManager::getName(myLetter));

// Test 2: Von Name zu Enum
const char* testInput = "F";
LetterManager::Letter found = LetterManager::getEnum(testInput);

if (found != LetterManager::UNKNOWN) {
    Serial.print(F("String "));
    Serial.print(testInput);
    Serial.print(F("' gefunden bei Index: "));
    Serial.println((int)found);
} else {
    Serial.println(F("String nicht gefunden!"));
}
}

void loop() {}
```

Verwende Code mit Vorsicht.

Details zur Suchfunktion:

- **strcmp_P**: Diese Funktion ist entscheidend. Sie vergleicht einen normalen String aus dem RAM (deine Sucheingabe) mit einem String, der im Flash (**_P** für Program Space) liegt.
- **UNKNOWN**: Ich habe einen **UNKNOWN**-Wert mit **-1** hinzugefügt, damit die Funktion sauber signalisieren kann, wenn ein String nicht in der Liste enthalten ist.
- **Effizienz**: Die Suche läuft linear durch die Tabelle. Bei nur 8 Einträgen (A-H) ist das extrem schnell.

Soll die Suche **case-insensitive** sein (also "a" findet auch "A"), oder reicht der exakte Vergleich aus?

KI-Antworten können Fehler enthalten. [Weitere Informationen](#)